NAME: PAVAN KUMAR S

REGISTER NO: 192312420

CROUSE CODE: ITA0613

SUBJECT NAME: MACHINE LEARNING

EXPERIMENT-02

FOR A GIVEN SET OF TRAINING DATA EXAMPLES STORED IN A .CSV FILE, IMPLEMENT AND DEMONSTRATE THE CANDIDATE-ELIMINATION ALGORITHM IN PYTHON TO OUTPUT A DESCRIPTION OF THE SET OF ALL HYPOTHESES CONSISTENT WITH THE TRAINING EXAMPLES

1: CANDIDATE-ELIMINATION FOR ENJOY SPORT

AIM

To implement Candidate-Elimination in Python using a CSV file and determine the version-space boundaries for the Enjoy Sport dataset.

APPARATUS / SOFTWARE REQUIRED

Computer or laptop, Python 3.x, IDLE/Jupyter Notebook/any Python IDE, and a text editor or spreadsheet application for creating CSV files.

PROCEDURE

1)Create a CSV file named enjoy_sport.csv using the dataset below.

2)Save the CSV file in the same folder as candidate_elimination.py.

3)Set the final line of the program to candidate_elimination('enjoy_sport.csv').

4)Run the program. It reads each example and updates For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples S and G based on the Yes/No target label.

5)Observe the Specific Boundary S and General Boundary G printed by the program.

DATASET DESCRIPTION

The attributes are Sky, AirTemp, Humidity, Wind, Water, and Forecast. Enjoy Sport is the target column. Yes rows are positive examples and No rows are negative examples.

Sky	AirTemp	Humidity	Wind	Water	Forecast	Enjoy Sport
Sunny	Warm	Normal	Strong	Warm	Same	Yes

Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

PROGRAM

```
import csv
def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))
def more_general_or_equal(h1, h2):
    return all(a == '?' or a == b or b == 'Ø'
               for a, b in zip(h1, h2))

def minimal_generalization(h, x):
    out = list(h)
    for i, value in enumerate(h):
        if value == 'Ø':
            out[i] = x[i]
        elif value != x[i] and value != '?':
            out[i] = '?'
    return out
def minimal_specializations(g, x, domains, S):
    result = []
    for i, value in enumerate(g):
        if value == '?':
            for v in domains[i]:
                if v != x[i]:
                    h = list(g)
                    h[i] = v
                    if any(more_general_or_equal(h, s) for s in S):
                        result.append(h)
    unique = []
    for h in result:
```

```
        if h not in unique:
            unique.append(h)
    return unique
def remove_redundant(G, S):
    G2 = [g for g in G if any(more_general_or_equal(g, s) for s in S)]
    G2 = [g for g in G2 if not any(
        g != other and more_general_or_equal(g, other) for other in G2)]
    S2 = [s for s in S if not any(
        s != other and more_general_or_equal(other, s) for other in S)]
    return S2, G2
def candidate_elimination(filename):
    with open(filename, newline='', encoding='utf-8-sig') as f:
        reader = csv.DictReader(f)
        rows = list(reader)
        columns = reader.fieldnames
    attributes, target = columns[:-1], columns[-1]
    X = [[row[a].strip() for a in attributes] for row in rows]
    y = [row[target].strip().lower() for row in rows]
    domains = [sorted(set(x[i] for x in X)) for i in range(len(attributes))]
    S = [['Ø'] * len(attributes)]
    G = [['?'] * len(attributes)]
    for x, label in zip(X, y):
        if label == 'yes':
            G = [g for g in G if covers(g, x)]
            new_S = []
            for s in S:
                if covers(s, x):
                    new_S.append(s)
                else:
                    h = minimal_generalization(s, x)
                    if any(more_general_or_equal(g, h) for g in G):
                        new_S.append(h)
            S = new_S
```

```
        elif label == 'no':
            S = [s for s in S if not covers(s, x)]
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)
                else:
                    new_G.extend(minimal_specializations(g, x, domains, S))
            G = new_G
        S, G = remove_redundant(G, S)
    print('Attributes:', attributes)
    print('Specific boundary S:')
    for h in S:
        print(h)
    print('General boundary G:')
    for h in G:
        print(h)
    print('S and G are the boundaries of the version space, not a list of every hypothesis.')


if __name__ == '__main__':
    candidate_elimination('enjoy_sport.csv')
```

INPUT

Save the dataset table above as enjoy_sport.csv. Keep the header names unchanged and ensure the last column contains only Yes or No.

OUTPUT

Attributes: ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']

Specific boundary S:

[Hypothesis or hypotheses printed by the program]

General boundary G:

[Hypothesis or hypotheses printed by the program]

RESULT

The CSV contains six descriptive attributes and one target label. The S and G boundaries delimit the hypotheses that remain consistent with all examples under the conjunctive hypothesis representation.

2: CANDIDATE-ELIMINATION FOR PLAY CRICKET

AIM

To implement Candidate-Elimination in Python using a CSV file and determine the version-space boundaries for predicting whether a person will play cricket.

APPARATUS / SOFTWARE REQUIRED

Computer or laptop, Python 3.x, IDLE/Jupyter Notebook/any Python IDE, and a text editor or spreadsheet application for creating CSV files.

PROCEDURE

1)Create a CSV file named play_cricket.csv using the dataset below.

2)Save the CSV file in the same folder as candidate_elimination.py.

3)Set the final line of the program to candidate_elimination('play_cricket.csv').

4)Run the program. It reads each example and updates S and G based on the Yes/No target label.

5)Observe the Specific Boundary S and General Boundary G printed by the program.

DATASET DESCRIPTION

The attributes are Weather, Temperature, Humidity, and Wind. Play Cricket is the target column. Both positive and negative examples are used to update the boundaries.

Weather	Temperature	Humidity	Wind	Play Cricket
Sunny	Hot	High	Weak	Yes
Sunny	Hot	High	Strong	Yes
Rainy	Cool	Normal	Weak	No
Sunny	Mild	High	Weak	Yes

PROGRAM

```
import csv
def covers(h, x):
   return all(a == '?' or a == b for a, b in zip(h, x))
def more_general_or_equal(h1, h2):
   return all(a == '?' or a == b or b == 'Ø'
```

```
            for a, b in zip(h1, h2))
def minimal_generalization(h, x):
    out = list(h)
    for i, value in enumerate(h):
        if value == 'Ø':
            out[i] = x[i]
        elif value != x[i] and value != '?':
            out[i] = '?'
    return out
def minimal_specializations(g, x, domains, S):
    result = []
    for i, value in enumerate(g):
        if value == '?':
            for v in domains[i]:
                if v != x[i]:
                    h = list(g)
                    h[i] = v
                    if any(more_general_or_equal(h, s) for s in S):
                        result.append(h)
    unique = []
    for h in result:
        if h not in unique:
            unique.append(h)
    return unique
def remove_redundant(G, S):
    G2 = [g for g in G if any(more_general_or_equal(g, s) for s in S)]
    G2 = [g for g in G2 if not any(
        g != other and more_general_or_equal(g, other) for other in G2)]
    S2 = [s for s in S if not any(
        s != other and more_general_or_equal(other, s) for other in S)]
    return S2, G2
def candidate_elimination(filename):
    with open(filename, newline='', encoding='utf-8-sig') as f:
```

```
    reader = csv.DictReader(f)
    rows = list(reader)
    columns = reader.fieldnames
attributes, target = columns[:-1], columns[-1]
X = [[row[a].strip() for a in attributes] for row in rows]
y = [row[target].strip().lower() for row in rows]
domains = [sorted(set(x[i] for x in X)) for i in range(len(attributes))]
S = [['Ø'] * len(attributes)]
G = [['?'] * len(attributes)]
for x, label in zip(X, y):
    if label == 'yes':
        G = [g for g in G if covers(g, x)]
        new_S = []
        for s in S:
            if covers(s, x):
                new_S.append(s)
            else:
                h = minimal_generalization(s, x)
                if any(more_general_or_equal(g, h) for g in G):
                    new_S.append(h)
        S = new_S
    elif label == 'no':
        S = [s for s in S if not covers(s, x)]
        new_G = []
        for g in G:
            if not covers(g, x):
                new_G.append(g)
            else:
                new_G.extend(minimal_specializations(g, x, domains, S))
        G = new_G
    S, G = remove_redundant(G, S)
print('Attributes:', attributes)
print('Specific boundary S:')
```

```
    for h in S:
        print(h)
    print('General boundary G:')
    for h in G:
        print(h)
    print('S and G are the boundaries of the version space, not a list of every hypothesis.')
if __name__ == '__main__':
    candidate_elimination('enjoy_sport.csv')
```

INPUT

Save the dataset table above as play_cricket.csv. Keep the header names unchanged and ensure the last column contains only Yes or No.

OUTPUT

Attributes: ['Weather', 'Temperature', 'Humidity', 'Wind']

Specific boundary S:

[Hypothesis or hypotheses printed by the program]

General boundary G:

[Hypothesis or hypotheses printed by the program]

RESULT

The CSV describes weather conditions and whether cricket is played. Positive examples generalize S; negative examples specialize G.

3: CANDIDATE-ELIMINATION FOR STUDENT PASS

AIM

To implement Candidate-Elimination in Python to identify the version-space boundaries for predicting whether a student will pass an examination.

APPARATUS / SOFTWARE REQUIRED

Computer or laptop, Python 3.x, IDLE/Jupyter Notebook/any Python IDE, and a text editor or spreadsheet application for creating CSV files.

PROCEDURE

1)Create a CSV file named student_pass.csv using the dataset below.

2)Save the CSV file in the same folder as candidate_elimination.py.

3)Set the final line of the program to candidate_elimination('student_pass.csv').

4)Run the program. It reads each example and updates S and G based on the Yes/No target label.

5)Observe the Specific Boundary S and General Boundary G printed by the program.

DATASET DESCRIPTION

The attributes are Study Hours, Attendance, Assignment, and Sleep. Pass is the target column, with Yes and No class labels.

Study Hours	Attendance	Assignment	Sleep	Pass
High	Good	Complete	Good	Yes
High	Good	Complete	Poor	Yes
Low	Poor	Incomplete	Good	No
High	Good	Incomplete	Good	Yes

PROGRAM

```
import csv


def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))
```

```
def more_general_or_equal(h1, h2):
    return all(a == '?' or a == b or b == 'Ø'
               for a, b in zip(h1, h2))
def minimal_generalization(h, x):
    out = list(h)
    for i, value in enumerate(h):
        if value == 'Ø':
            out[i] = x[i]
        elif value != x[i] and value != '?':
            out[i] = '?'
    return out
def minimal_specializations(g, x, domains, S):
    result = []
    for i, value in enumerate(g):
        if value == '?':
            for v in domains[i]:
                if v != x[i]:
                    h = list(g)
                    h[i] = v
                    if any(more_general_or_equal(h, s) for s in S):
                        result.append(h)
    unique = []
    for h in result:
        if h not in unique:
            unique.append(h)
    return unique
def remove_redundant(G, S):
    G2 = [g for g in G if any(more_general_or_equal(g, s) for s in S)]
    G2 = [g for g in G2 if not any(
        g != other and more_general_or_equal(g, other) for other in G2)]
    S2 = [s for s in S if not any(
        s != other and more_general_or_equal(other, s) for other in S)]
    return S2, G2
```

```
def candidate_elimination(filename):
    with open(filename, newline='', encoding='utf-8-sig') as f:
        reader = csv.DictReader(f)
        rows = list(reader)
        columns = reader.fieldnames
    attributes, target = columns[:-1], columns[-1]
    X = [[row[a].strip() for a in attributes] for row in rows]
    y = [row[target].strip().lower() for row in rows]
    domains = [sorted(set(x[i] for x in X)) for i in range(len(attributes))]
    S = [['Ø'] * len(attributes)]
    G = [['?'] * len(attributes)]
    for x, label in zip(X, y):
        if label == 'yes':
            G = [g for g in G if covers(g, x)]
            new_S = []
            for s in S:
                if covers(s, x):
                    new_S.append(s)
                else:
                    h = minimal_generalization(s, x)
                    if any(more_general_or_equal(g, h) for g in G):
                        new_S.append(h)
            S = new_S
        elif label == 'no':
            S = [s for s in S if not covers(s, x)]
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)
                else:
                    new_G.extend(minimal_specializations(g, x, domains, S))
            G = new_G
```

```
        S, G = remove_redundant(G, S)
    print('Attributes:', attributes)
    print('Specific boundary S:')
    for h in S:
        print(h)
    print('General boundary G:')
    for h in G:
        print(h)
    print('S and G are the boundaries of the version space, not a list of every hypothesis.')
if __name__ == '__main__':
    candidate_elimination('enjoy_sport.csv')
```

INPUT

Save the dataset table above as student_pass.csv. Keep the header names unchanged and ensure the last column contains only Yes or No.

OUTPUT

Attributes: ['Study Hours', 'Attendance', 'Assignment', 'Sleep']

Specific boundary S:

[Hypothesis or hypotheses printed by the program]

General boundary G:

[Hypothesis or hypotheses printed by the program]

RESULT

The dataset relates student study conditions to a pass/fail label. The boundaries summarize hypotheses consistent with the recorded examples

4: CANDIDATE-ELIMINATION FOR LAPTOP PURCHASE

AIM

To implement Candidate-Elimination in Python using a CSV file and find the version-space boundaries for predicting whether a customer will purchase a laptop.

APPARATUS / SOFTWARE REQUIRED

Computer or laptop, Python 3.x, IDLE/Jupyter Notebook/any Python IDE, and a text editor or spreadsheet application for creating CSV files.

PROCEDURE

1)Create a CSV file named laptop_purchase.csv using the dataset below.

2)Save the CSV file in the same folder as candidate_elimination.py.

3)Set the final line of the program to candidate_elimination('laptop_purchase.csv').

4)Run the program. It reads each example and updates S and G based on the Yes/No target label.

5)Observe the Specific Boundary S and General Boundary G printed by the program.

DATASET DESCRIPTION

The attributes are Price, Brand, RAM, and Storage. Purchase is the target column. The examples include both Yes and No labels.

Price	Brand	RAM	Storage	Purchase
Low	A	8GB	SSD	Yes
Low	A	16GB	SSD	Yes
High	B	8GB	HDD	No
Low	A	8GB	HDD	Yes

PROGRAM

```
import csv
def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x))
def more_general_or_equal(h1, h2):
    return all(a == '?' or a == b or b == 'Ø'
```

```
            for a, b in zip(h1, h2))
def minimal_generalization(h, x):
    out = list(h)
    for i, value in enumerate(h):
        if value == 'Ø':
            out[i] = x[i]
        elif value != x[i] and value != '?':
            out[i] = '?'
    return out
def minimal_specializations(g, x, domains, S):
    result = []
    for i, value in enumerate(g):
        if value == '?':
            for v in domains[i]:
                if v != x[i]:
                    h = list(g)
                    h[i] = v
                    if any(more_general_or_equal(h, s) for s in S):
                        result.append(h)
    unique = []
    for h in result:
        if h not in unique:
            unique.append(h)
    return unique
def remove_redundant(G, S):
    G2 = [g for g in G if any(more_general_or_equal(g, s) for s in S)]
    G2 = [g for g in G2 if not any(
        g != other and more_general_or_equal(g, other) for other in G2)]
    S2 = [s for s in S if not any(
        s != other and more_general_or_equal(other, s) for other in S)]
    return S2, G2
def candidate_elimination(filename):
    with open(filename, newline='', encoding='utf-8-sig') as f:
```

```
    reader = csv.DictReader(f)
    rows = list(reader)
    columns = reader.fieldnames
attributes, target = columns[:-1], columns[-1]
X = [[row[a].strip() for a in attributes] for row in rows]
y = [row[target].strip().lower() for row in rows]
domains = [sorted(set(x[i] for x in X)) for i in range(len(attributes))]
S = [['Ø'] * len(attributes)]
G = [['?'] * len(attributes)]
for x, label in zip(X, y):
    if label == 'yes':
        G = [g for g in G if covers(g, x)]
        new_S = []
        for s in S:
            if covers(s, x):
                new_S.append(s)
            else:
                h = minimal_generalization(s, x)
                if any(more_general_or_equal(g, h) for g in G):
                    new_S.append(h)
        S = new_S
    elif label == 'no':
        S = [s for s in S if not covers(s, x)]
        new_G = []
        for g in G:
            if not covers(g, x):
                new_G.append(g)
            else:
                new_G.extend(minimal_specializations(g, x, domains, S))
        G = new_G
    S, G = remove_redundant(G, S)
print('Attributes:', attributes)
print('Specific boundary S:')
```

```
    for h in S:
        print(h)
    print('General boundary G:')
    for h in G:
        print(h)
    print('S and G are the boundaries of the version space, not a list of every hypothesis.')
if __name__ == '__main__':
    candidate_elimination('enjoy_sport.csv')
```

INPUT

Save the dataset table above as laptop_purchase.csv. Keep the header names unchanged and ensure the last column contains only Yes or No.

OUTPUT

Attributes: ['Price', 'Brand', 'RAM', 'Storage']

Specific boundary S:

[Hypothesis or hypotheses printed by the program]

General boundary G:

[Hypothesis or hypotheses printed by the program]

RESULT

The dataset describes laptop features and customer purchase labels. The program updates both boundaries from the training rows.

5: CANDIDATE-ELIMINATION FOR OUTDOOR ACTIVITY

AIM

To implement Candidate-Elimination in Python using a CSV file and determine the version-space boundaries for predicting whether an outdoor activity will take place.

APPARATUS / SOFTWARE REQUIRED

Computer or laptop, Python 3.x, IDLE/Jupyter Notebook/any Python IDE, and a text editor or spreadsheet application for creating CSV files.

PROCEDURE

1)Create a CSV file named outdoor_activity.csv using the dataset below.

2)Save the CSV file in the same folder as candidate_elimination.py.

3)Set the final line of the program to candidate_elimination('outdoor_activity.csv').

4)Run the program. It reads each example and updates S and G based on the Yes/No target label.

5) Observe the Specific Boundary S and General Boundary G printed by the program.

DATASET DESCRIPTION

The attributes are Weather, Temperature, Wind, and Forecast. Activity is the target column, with Yes and No labels.

Weather	Temperature	Wind	Forecast	Activity
Sunny	Warm	Weak	Clear	Yes
Sunny	Warm	Strong	Clear	Yes
Rainy	Cold	Strong	Cloudy	No
Sunny	Cold	Weak	Clear	Yes

PROGRAM

```
import csv
def covers(h, x):
    return all(a == '?' or a == b for a, b in zip(h, x)
def more_general_or_equal(h1, h2):
    return all(a == '?' or a == b or b == 'Ø'
```

```
            for a, b in zip(h1, h2))
def minimal_generalization(h, x):
    out = list(h)
    for i, value in enumerate(h):
        if value == 'Ø':
            out[i] = x[i]
        elif value != x[i] and value != '?':
            out[i] = '?'
    return out
 def minimal_specializations(g, x, domains, S):
    result = []
    for i, value in enumerate(g):
        if value == '?':
            for v in domains[i]:
                if v != x[i]:
                    h = list(g)
                    h[i] = v
                    if any(more_general_or_equal(h, s) for s in S):
                        result.append(h)
    unique = []
    for h in result:
        if h not in unique:
            unique.append(h)
    return unique
def remove_redundant(G, S):
    G2 = [g for g in G if any(more_general_or_equal(g, s) for s in S)]
    G2 = [g for g in G2 if not any(
        g != other and more_general_or_equal(g, other) for other in G2)]
    S2 = [s for s in S if not any(
        s != other and more_general_or_equal(other, s) for other in S)]
    return S2, G2
def candidate_elimination(filename):
    with open(filename, newline='', encoding='utf-8-sig') as f:
```

```
    reader = csv.DictReader(f)
    rows = list(reader)
    columns = reader.fieldnames
attributes, target = columns[:-1], columns[-1]
X = [[row[a].strip() for a in attributes] for row in rows]
y = [row[target].strip().lower() for row in rows]
domains = [sorted(set(x[i] for x in X)) for i in range(len(attributes))]
S = [['Ø'] * len(attributes)]
G = [['?'] * len(attributes)]
for x, label in zip(X, y):
    if label == 'yes':
        G = [g for g in G if covers(g, x)]
        new_S = []
        for s in S:
            if covers(s, x):
                new_S.append(s)
            else:
                h = minimal_generalization(s, x)
                if any(more_general_or_equal(g, h) for g in G):
                    new_S.append(h)
        S = new_S
    elif label == 'no':
        S = [s for s in S if not covers(s, x)]
        new_G = []
        for g in G:
            if not covers(g, x):
                new_G.append(g)
            else:
                new_G.extend(minimal_specializations(g, x, domains, S))
        G = new_G
    S, G = remove_redundant(G, S)
print('Attributes:', attributes)
print('Specific boundary S:')
```

```
    for h in S:
        print(h)
    print('General boundary G:')
    for h in G:
        print(h)
    print('S and G are the boundaries of the version space, not a list of every hypothesis.')
if __name__ == '__main__':
    candidate_elimination('enjoy_sport.csv')
```

INPUT

Save the dataset table above as outdoor_activity.csv. Keep the header names unchanged and ensure the last column contains only Yes or No.

OUTPUT

Attributes: ['Weather', 'Temperature', 'Wind', 'Forecast']

Specific boundary S:

[Hypothesis or hypotheses printed by the program]

General boundary G:

[Hypothesis or hypotheses printed by the program]

RESULT

The dataset describes outdoor conditions and whether an activity takes place. S and G delimit the consistent version space.